MAPR

Spark GraphX

Learning Goals

- Describe GraphX
- Define Regular, Directed, and Property Graphs
- Create a Property Graph
- Perform Operations on Graphs

Learning Goals

- **Describe GraphX**
- Define Regular, Directed, and Property Graphs
- Create a Property Graph
- Perform Operations on Graphs

What is a Graph?

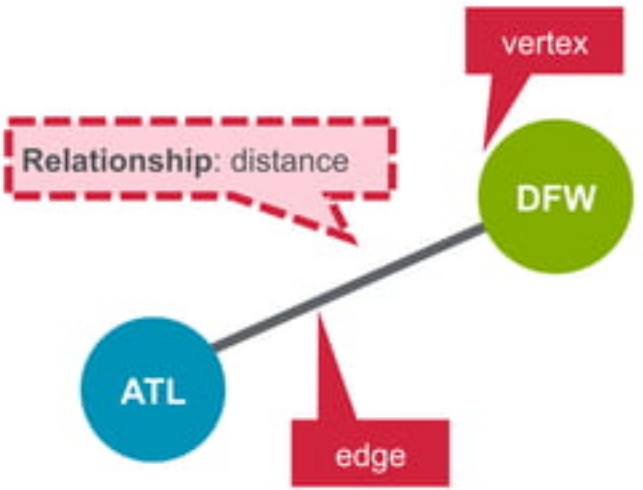
Graph: vertices connected by edges

$$G = (V, E)$$

Graph Vertices Edges

What is a Graph?

set of vertices, connected by edges.

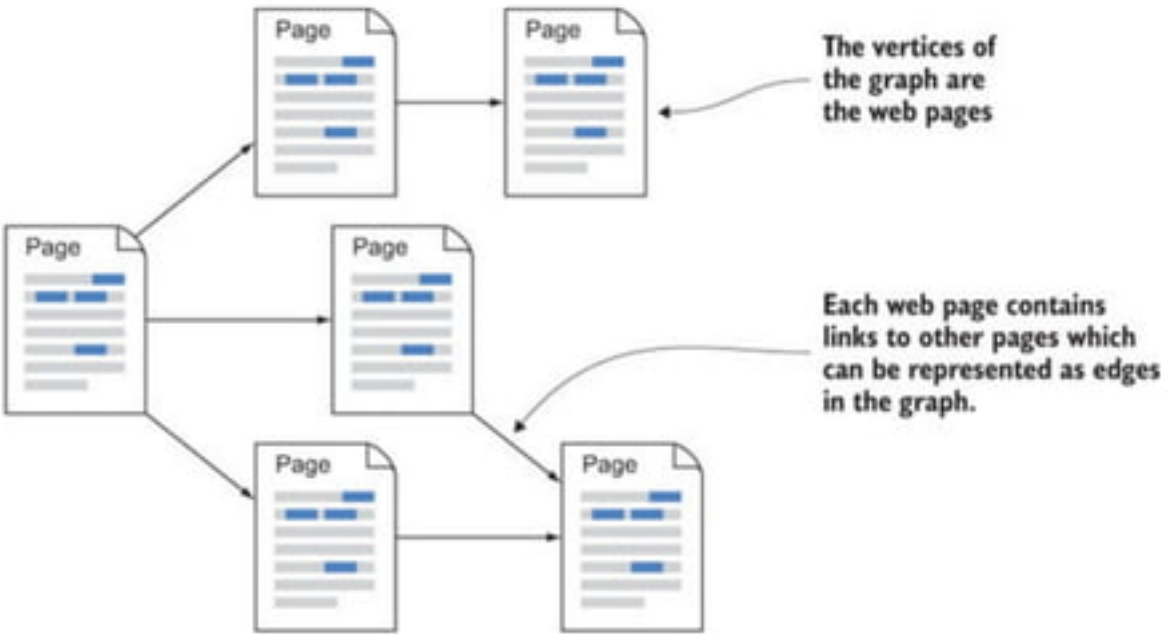


Graphs are Essential to Data Mining and Machine Learning

- Identify influential entities (people, information…)
- Find communities
- Understand people's shared interests
- Model complex data dependencies

Real World Graphs

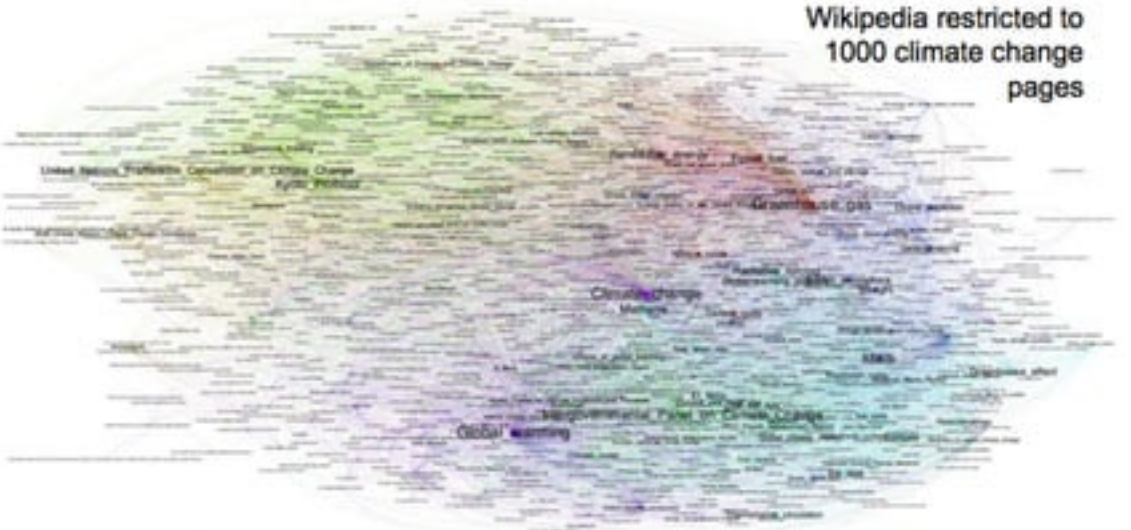
- Web Pages



Real World Graphs

- Web Pages

Web Graphs



- Vertices: *Web-pages*
- Edges: *Links (Directed)*

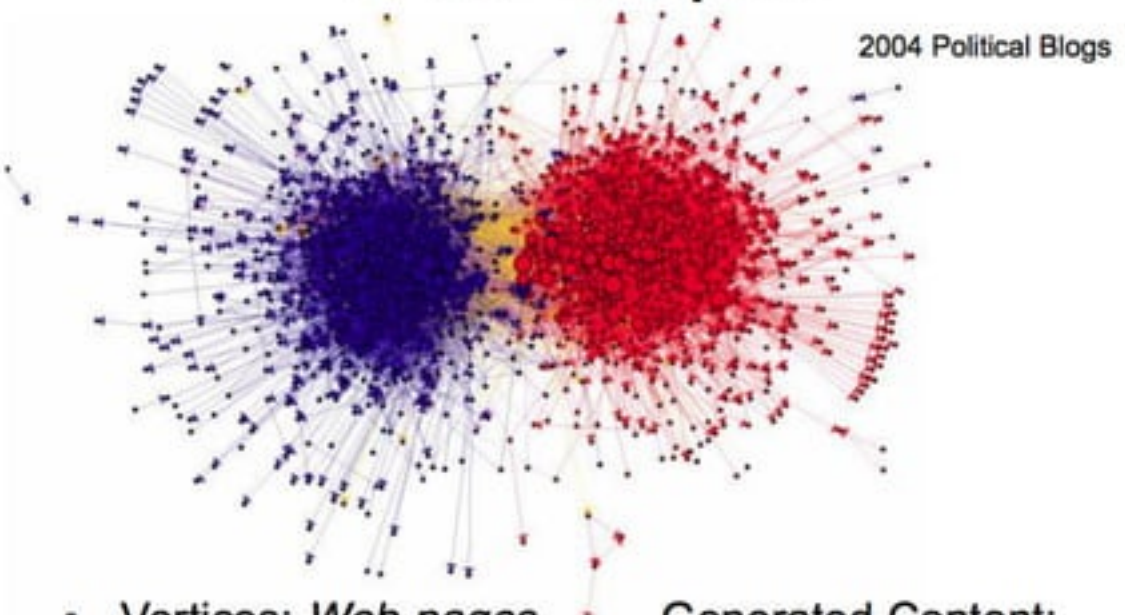
Generated Content:

- Click-streams

Real World Graphs

- Web Pages

Web Graphs



- Vertices: *Web-pages*
- Edges: *Links (Directed)*

Generated Content:

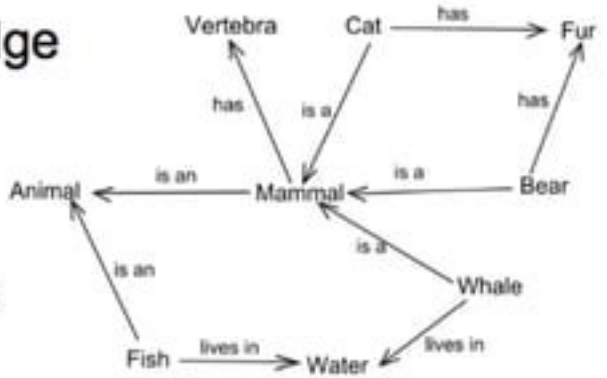
- Click-streams

Semantic Networks

Organize Knowledge

Vertices: Subject, Object

Edges: Predicates



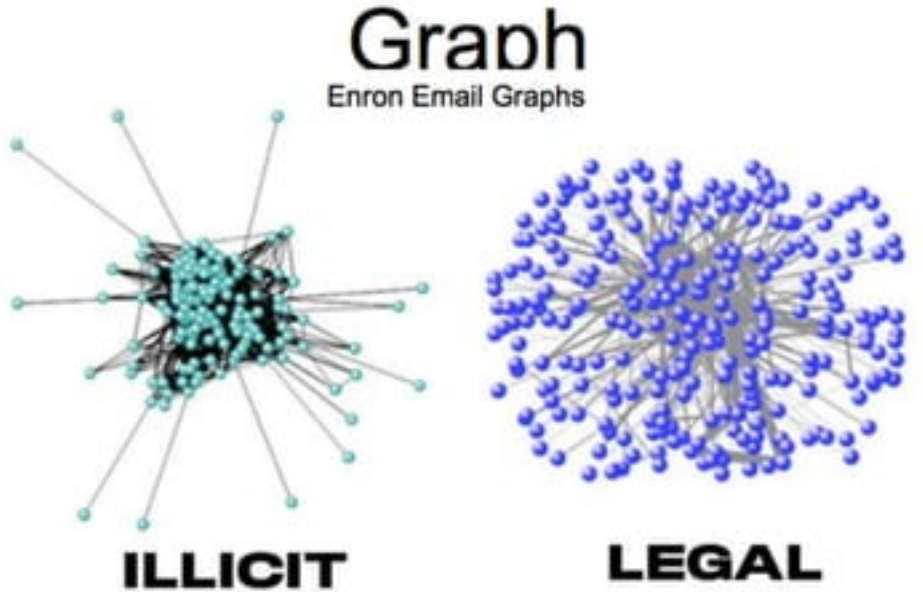
Example:

Google Knowledge Graph

- 570M Vertices
- 18B Edges

http://wiki.dbpedia.org

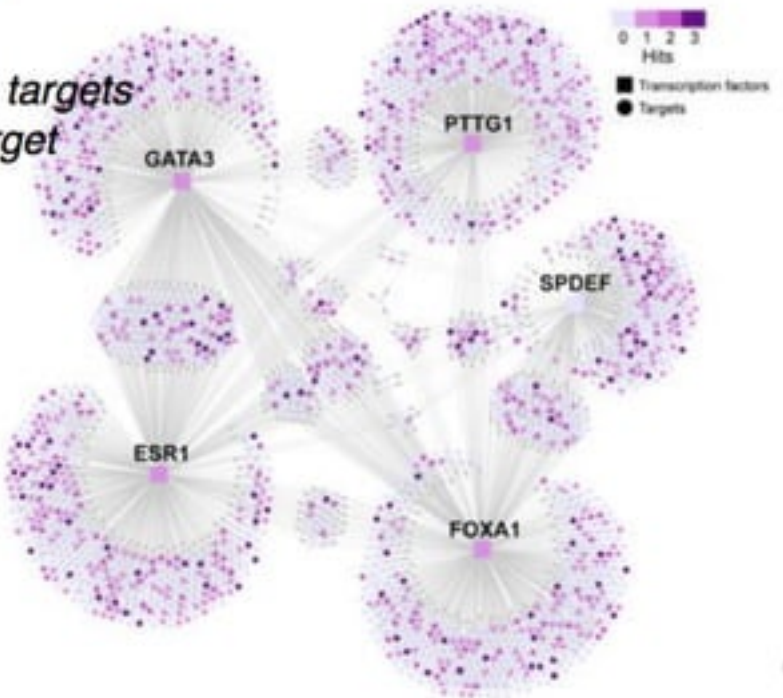
Real World Graphs



Vertices: *Users*
Directed Edges: *Email From→To*

Biological Networks

Regulatory Networks
(Bipartite)
Vertices: *Regulators, targets*
Edges: *Regulates target*

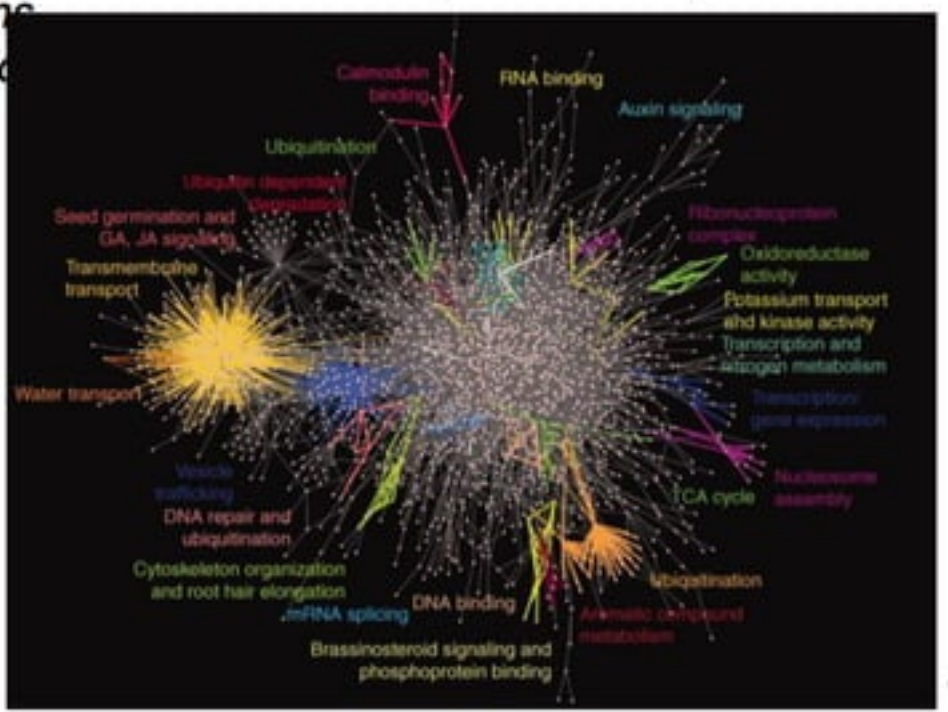


Biological Networks

Protein-Protein Interaction Networks (Interactomes)

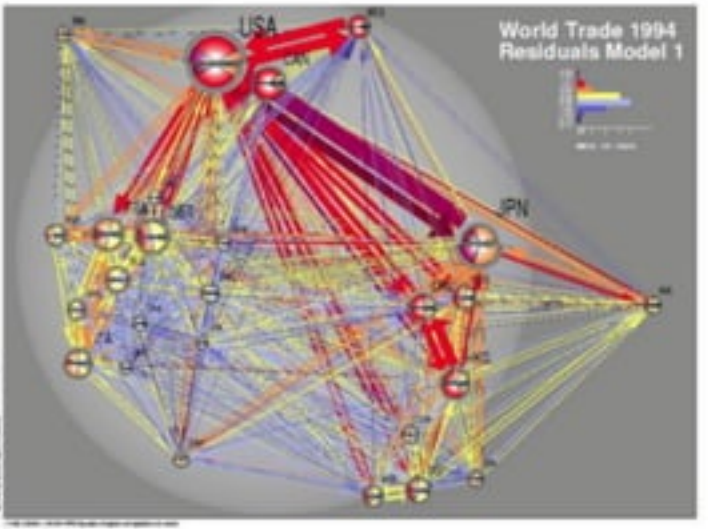
Vertices: *Proteins*

Edges: *Interactions*



Transaction Networks

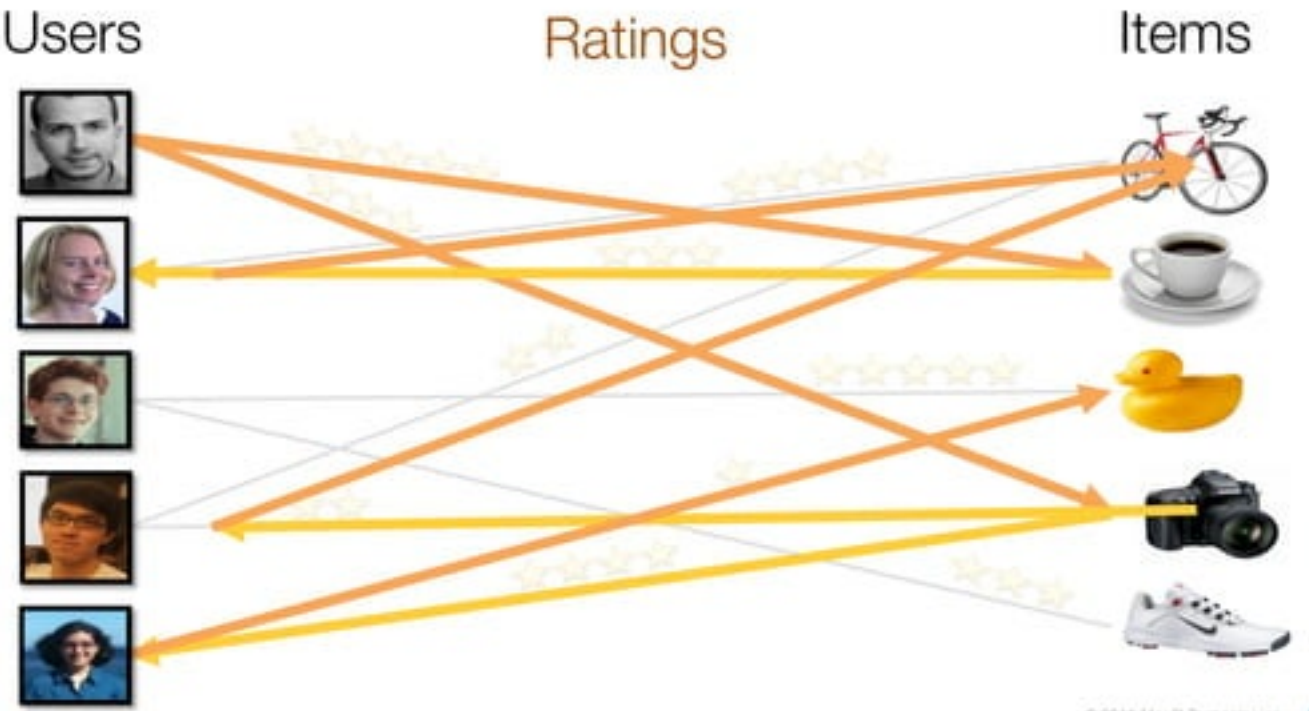
Supply Chain:
Vertices: Suppliers/Consumers
Edges: Exchange of Goods



Transaction Networks (e.g., Bitcoin):
Vertices: Users
Edges: Exchange of Currency

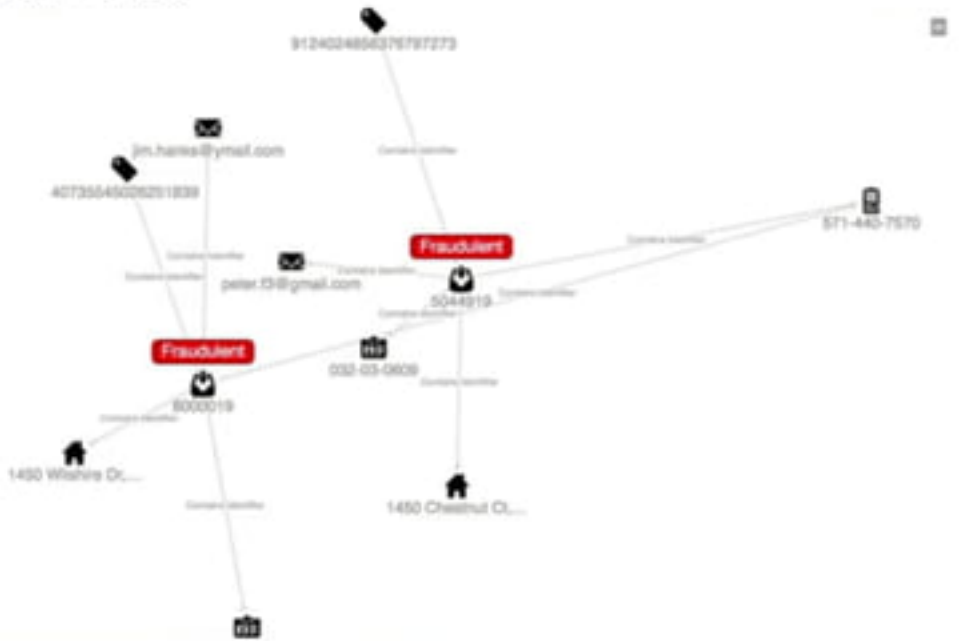
Real World Graphs

- Recommendations



Real World Graphs

- Credit Card Application Fraud

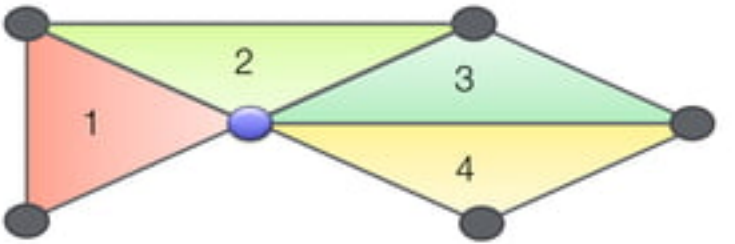


Real World Graphs

- Credit Card Fraud

Finding Communities

Count triangles passing through each vertex:



Measures “cohesiveness” of local community

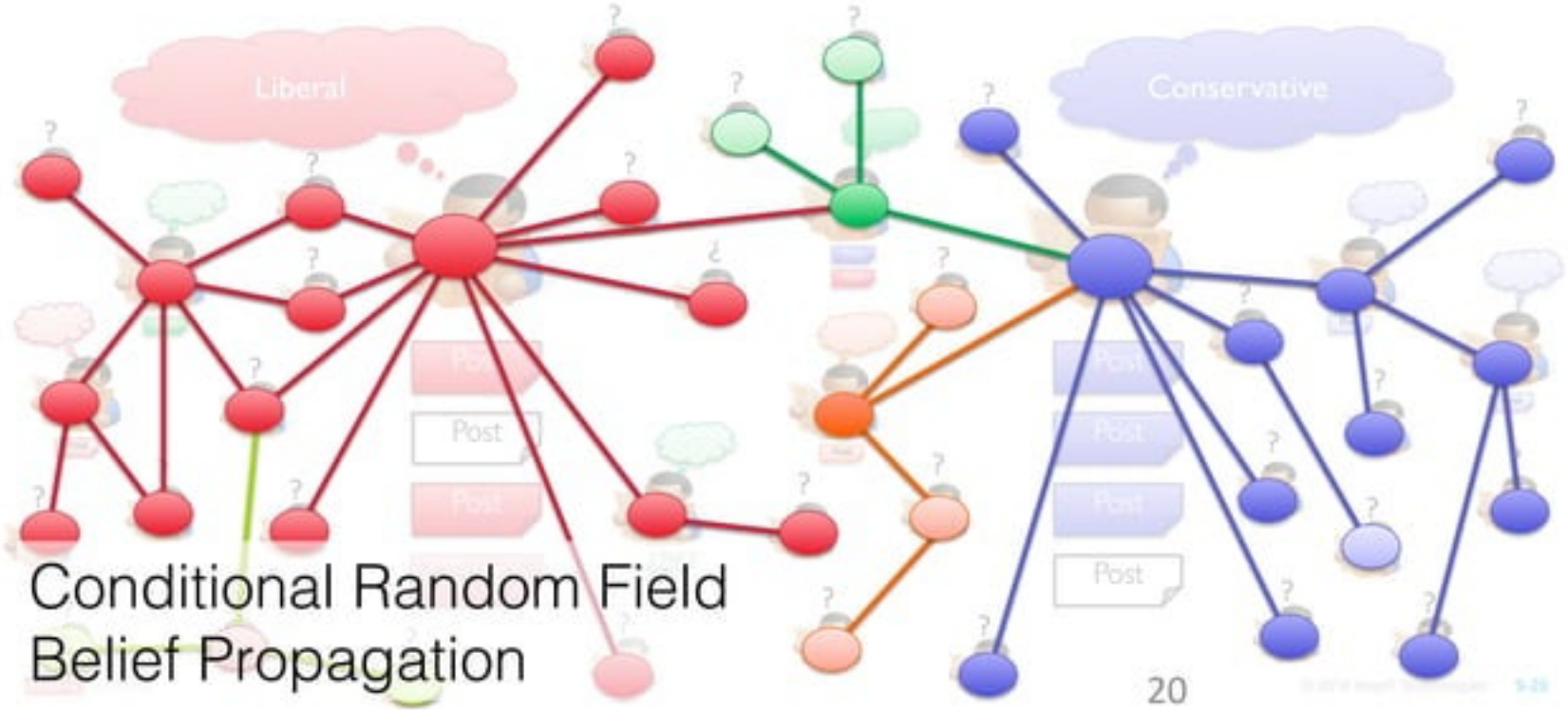
Fewer Triangles
Weaker Community

More Triangles
Stronger Community

Real World Graphs

Healthcare

Predicting User Behavior



Conditional Random Field
Belief Propagation

Enable Joining Tables and Graphs

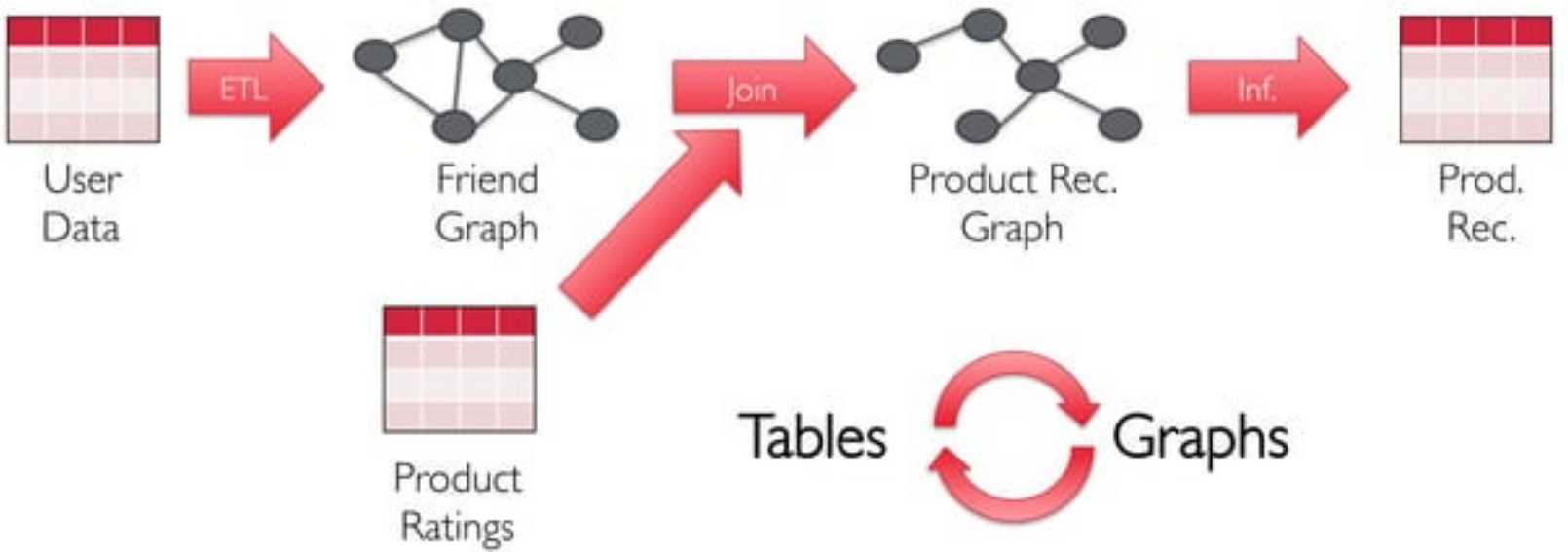
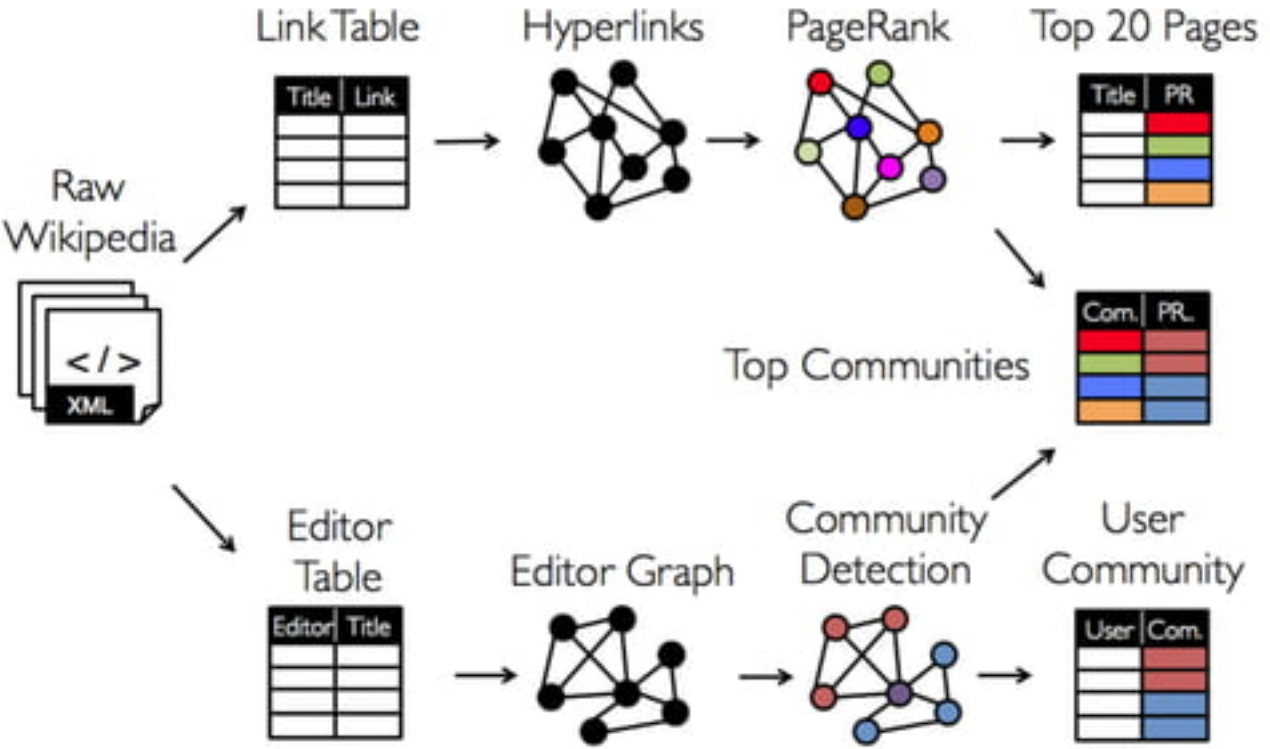
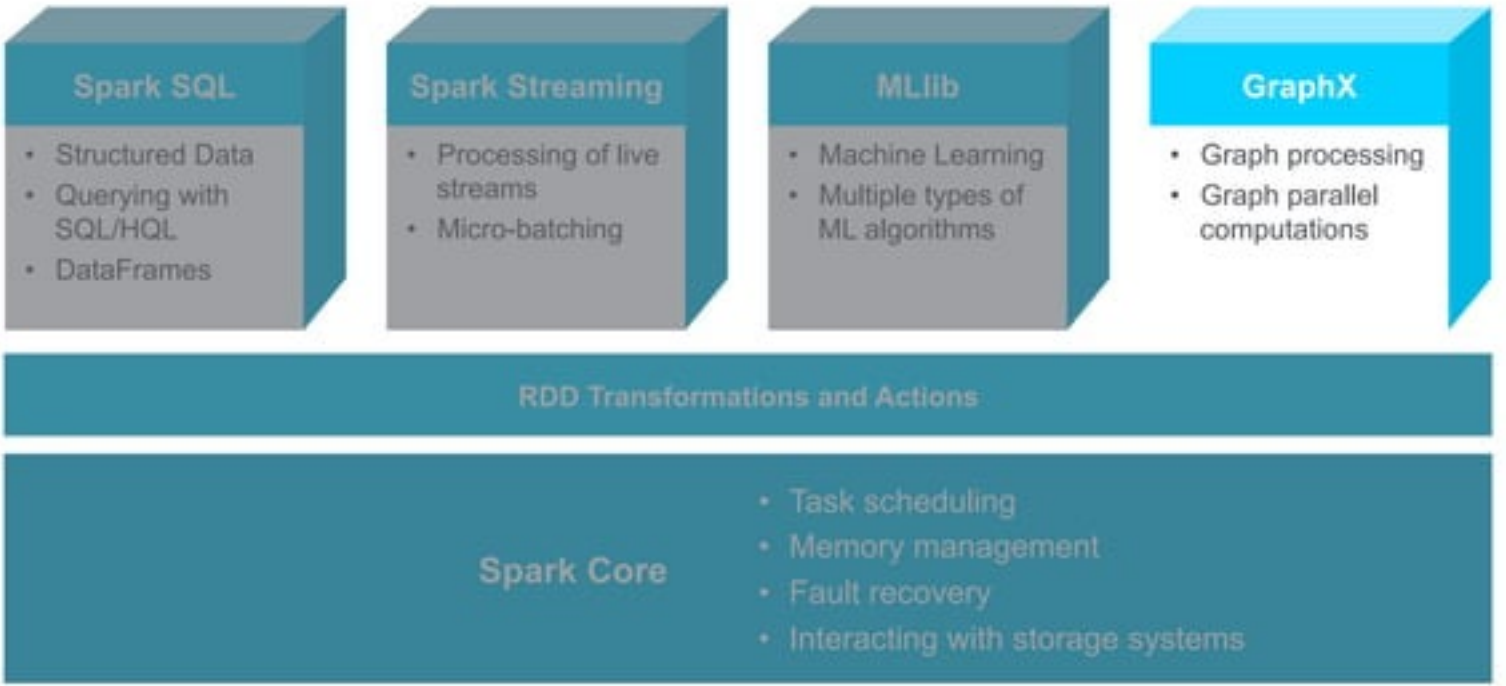


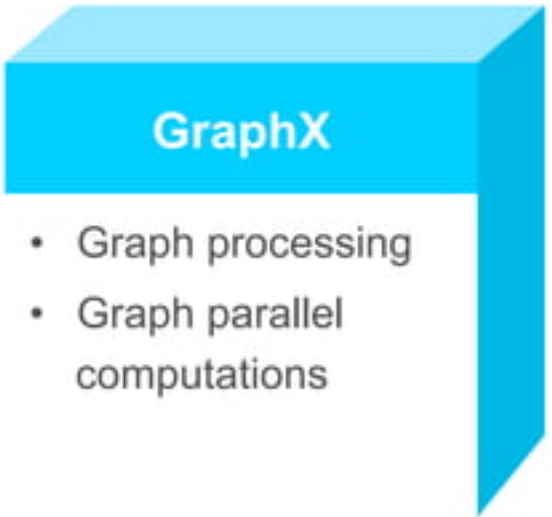
Table and Graph Analytics



What is GraphX?



Apache Spark GraphX



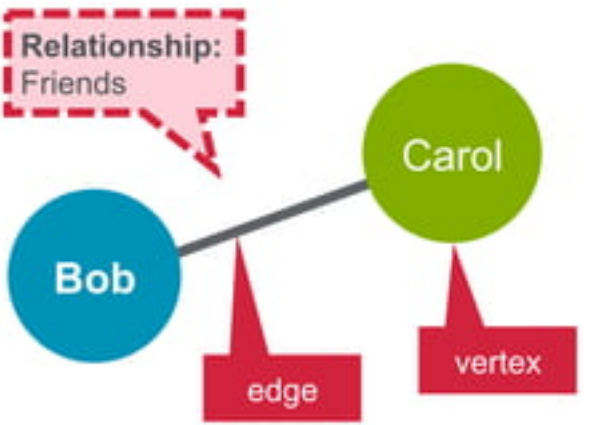
- Spark component for graphs and graph-parallel computations
- Combines data parallel and graph parallel processing in single API
- View data as graphs and as collections (RDD)
  - no duplication or movement of data
- Operations for graph computation
  - includes optimized version of Pregel
- Provides graph algorithms and builders

Learning Goals

- Describe GraphX
- **Define Regular, Directed, and Property Graphs**
- Create a Property Graph
- Perform Operations on Graphs

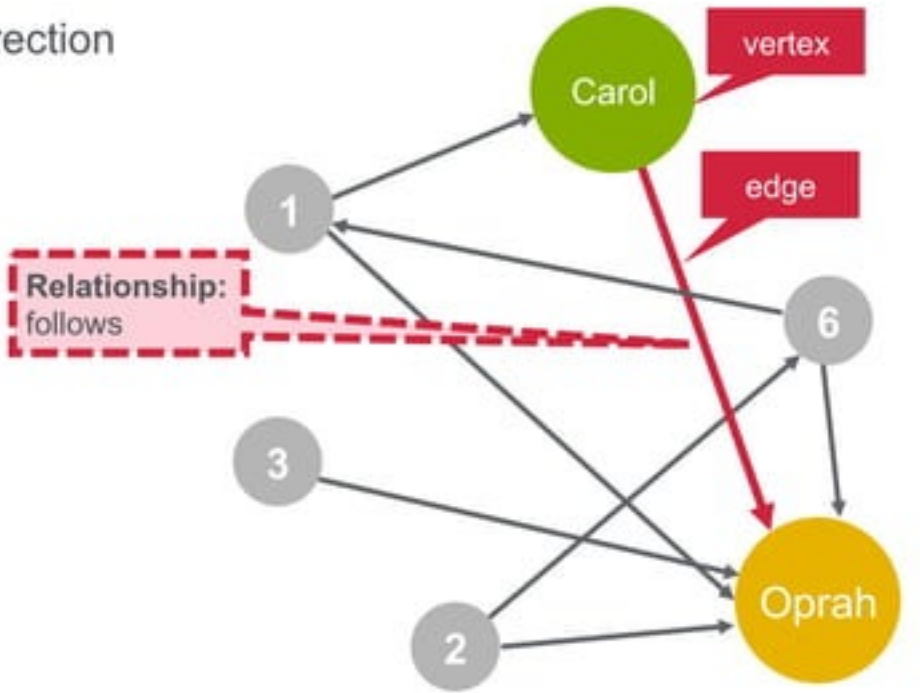
Regular Graphs vs Directed Graphs

- Regular graph: each vertex has the same number of edges
- Example: Facebook friends
  - Bob is a friend of Carol
  - Carol is a friend of Bob

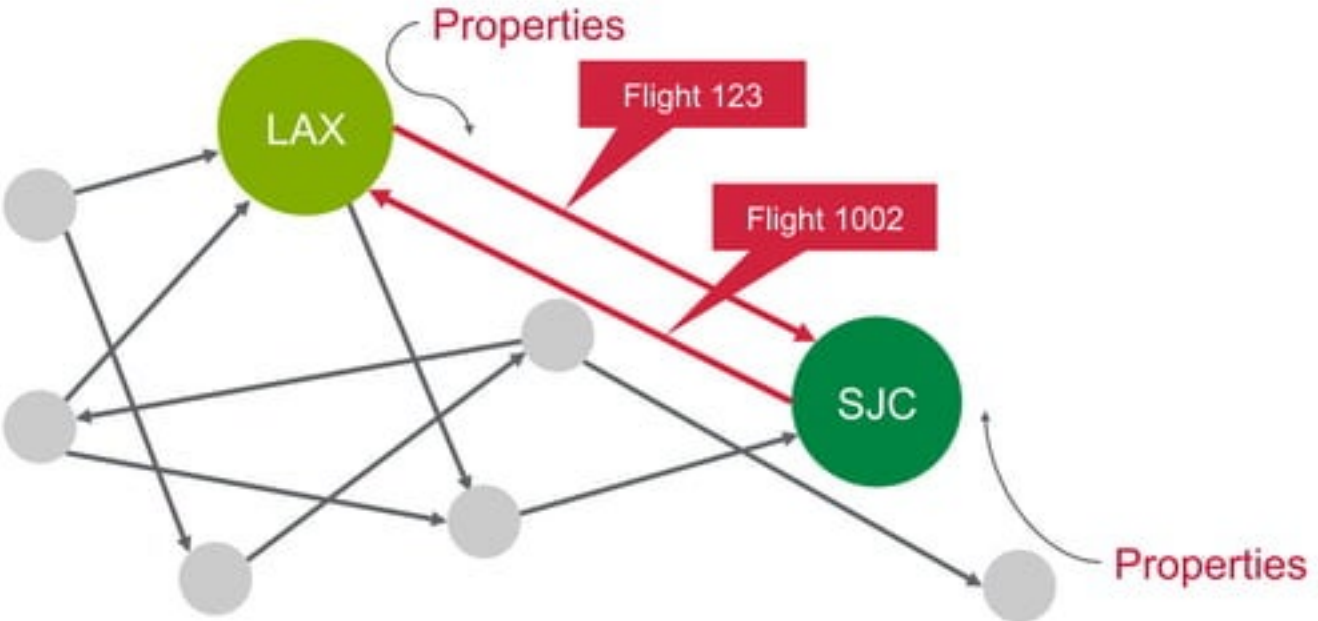


Regular Graphs vs Directed Graphs

- Directed graph: edges have a direction
- Example: Twitter followers
  - Carol follows Oprah
  - Oprah does not follow Carol

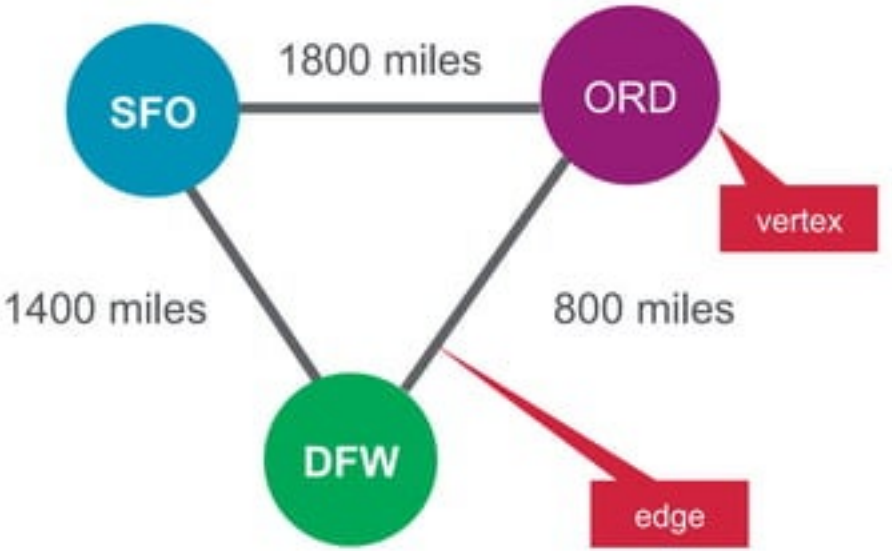


Property Graph



Flight Example with GraphX

Originating Airport	Destination Airport	Distance
SFO	ORD	1800 miles
ORD	DFW	800 miles
DFW	SFO	1400 miles



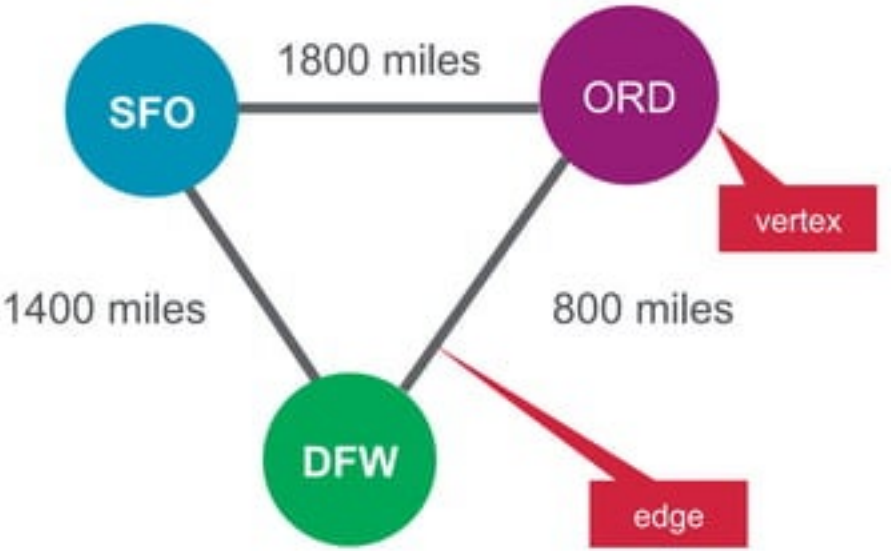
Flight Example with GraphX

Vertex Table

Id	Property
1	SFO
2	ORD
3	DFW

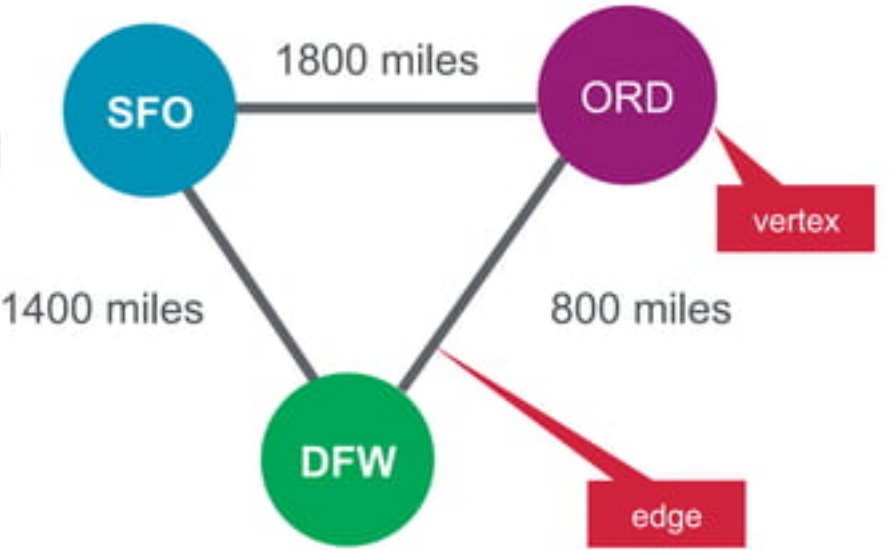
Edge Table

SrcId	DestId	Property
1	2	1800
2	3	800
3	1	1400



Spark Property Graph class

```
class Graph[VD, ED] {
  val vertices: VertexRDD[VD]
  val edges: EdgeRDD[ED]
}
```



Learning Goals

- Define GraphX
- Define Regular, Directed, and Property Graphs
- **Create a Property Graph**
- Perform Operations on Graphs

Create a Property Graph

1. Import required classes
2. Create vertex RDD
3. Create edge RDD
4. Create graph

Create a Property Graph

1. Import required classes

```
import org.apache.spark._
import org.apache.spark.graphx._
import org.apache.spark.rdd.RDD
```

Create a Property Graph: Data Set

Vertices: Airports

Vertex ID	Property (V)
Id (Long)	Name (String)

Edges: Routes

Source ID	Dest ID	Property (E)
Id	Id	Distance (Integer)

Create a Property Graph

Id	Property
1	SFO
2	ORD
3	DFW

2 Create vertex RDD

```
// create  vertices RDD with ID and Name
val vertices=Array((1L, ("SFO")),(2L, ("ORD")),(3L,("DFW")))
val vRDD= sc.parallelize(vertices)
vRDD.take(1)
// Array((1,SFO))
```

Create a Property Graph

SrcId	DestId	Property
1	2	1800
2	3	800
3	1	1400

3 Create edge RDD

```
// create  routes  RDD with srcid, destid , distance
val edges = Array(Edge(1L,2L,1800),Edge(2L,3L,800),
    Edge(3L,1L,1400))
val eRDD= sc.parallelize(edges)

eRDD.take(2)
// Array(Edge(1,2,1800), Edge(2,3,800))
```

Create a Property Graph

4 Create graph

```
// define default vertex nowhere
val nowhere = "nowhere"

//build initial graph
val graph = Graph(vertices, edges, nowhere)

graph.vertices.take(3).foreach(print)
//  (2,ORD)(1,SFO)(3,DFW)

graph.edges.take(3).foreach(print)
// Edge(1,2,1800) Edge(2,3,800) Edge(3,1,1400)
```

Learning Goals

- Define GraphX
- Define Regular, Directed, and Property Graphs
- Create a Property Graph
- **Perform Operations on Graphs**

Graph Operators

To answer questions such as:

- How many airports are there?
- How many flight routes are there?
- What are the longest distance routes?
- Which airport has the most incoming flights?
- What are the top 10 flights?

Graph Class

```scala
/** Summary of the functionality in the property graph */
class Graph[VD, ED] {
  // Information about the Graph ==========================================
  val numEdges: Long
  val numVertices: Long
  val inDegrees: VertexRDD[Int]
  val outDegrees: VertexRDD[Int]
  val degrees: VertexRDD[Int]
  // Views of the graph as collections ====================================
  val vertices: VertexRDD[VD]
  val edges: EdgeRDD[ED]
  val triplets: RDD[EdgeTriplet[VD, ED]]
```

Graph Operators

To find information about the graph

Operator	Description
`numEdges`	number of edges `(Long)`
`numVertices`	number of vertices `(Long)`
`inDegrees`	The in-degree of each vertex `(VertexRDD[Int])`
`outDegrees`	The out-degree of each vertex `(VertexRDD[Int])`
`degrees`	The degree of each vertex `(VertexRDD[Int])`

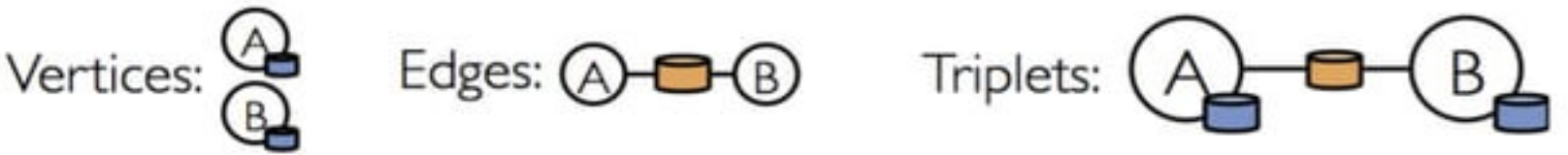
Graph Operators

Graph Operators

```
// How many airports?
val numairports = graph.numVertices
// Long = 3
// How many routes?
val numroutes = graph.numEdges
// Long = 3

// routes > 1000 miles distance?
graph.edges.filter {
    case ( Edge(org_id, dest_id,distance))=>
    distance > 1000
    }.take(3)
// Array(Edge(1,2,1800), Edge(3,1,1400)
```

Triplets




```
// Triplets add source and destination properties to Edges
graph.triplets.take(3).foreach(println)
((1,SFO),(2,ORD),1800)
((2,ORD),(3,DFW),800)
((3,DFW),(1,SFO),1400)
```

Triplets What are the longest routes ?

```
((1,SFO),(2,ORD),1800)
((2,ORD),(3,DFW),800)
((3,DFW),(1,SFO),1400)

// print out longest routes
graph.triplets.sortBy(_.attr, ascending=false)
 .map(triplet =>"Distance" + triplet.attr.toString + "from"
 + triplet.srcAttr + "to" + triplet.dstAttr)
 .collect.foreach(println)

Distance 1800 from SFO to ORD
Distance 1400 from DFW to SFO
Distance 800 from ORD to DFW
```

Graph Operators

Which airport has the most incoming flights? (real dataset)

```
// Define a function to compute the highest degree vertex
def max(a:(VertexId,Int),b:(VertexId, Int)):(VertexId, Int) =
{
  if (a._2 > b._2) a else b
}
// Which Airport has the most incoming flights?
val maxInDegree:(VertexId, Int)= graph.inDegrees.reduce(max)
// (10397,152)  ATL
```

Graph Operators

Which 3 airports have the most incoming flights? (real dataset)

```
// get top 3
val maxIncoming = graph.inDegrees.collect
 .sortWith(_._2 > _._2)
 .map(x => (airportMap(x._1), x._2)).take(3)

maxIncoming.foreach(println)
(ATL,152)
(ORD,145)
(DFW,143)
```

Graph Operators

Caching Graphs

Operator	Description
`cache()`	Caches the vertices and edges; default level is `MEMORY_ONLY`
`persist(newLevel)`	Caches the vertices and edges at specified storage level; returns a reference to this graph
`unpersist(blocking)`	Uncaches both vertices and edges of this graph
`unpersistVertices(blocking)`	Uncaches only the vertices, leaving edges alone

```
// Transform vertex and edge attributes ==========
def mapVertices[VD2](map: (VertexID, VD) => VD2): Graph[VD2, ED]
def mapEdges[ED2](map: Edge[ED] => ED2): Graph[VD, ED2]
def mapEdges[ED2](map: (PartitionID, Iterator[Edge[ED]]) => Iterator[ED2]): Graph[VD, ED2]
def mapTriplets[ED2](map: EdgeTriplet[VD, ED] => ED2): Graph[VD, ED2]
def mapTriplets[ED2](map: (PartitionID, Iterator[EdgeTriplet[VD, ED]]) => Iterator[ED2])
  : Graph[VD, ED2]
// Modify the graph structure ==========
def reverse: Graph[VD, ED]
def subgraph(
    epred: EdgeTriplet[VD,ED] => Boolean = (x => true),
    vpred: (VertexID, VD) => Boolean = ((v, d) => true))
  : Graph[VD, ED]
def mask[VD2, ED2](other: Graph[VD2, ED2]): Graph[VD, ED]
def groupEdges(merge: (ED, ED) => ED): Graph[VD, ED]
// Join RDDs with the graph ==========
def joinVertices[U](table: RDD[(VertexID, U)])(mapFunc: (VertexID, VD, U) => VD): Graph[VD, ED]
def outerJoinVertices[U, VD2](other: RDD[(VertexID, U)])
    (mapFunc: (VertexID, VD, Option[U]) => VD2)
  : Graph[VD2, ED]
// Aggregate information about adjacent triplets ==========
def collectNeighborIds(edgeDirection: EdgeDirection): VertexRDD[Array[VertexID]]
def collectNeighbors(edgeDirection: EdgeDirection): VertexRDD[Array[(VertexID, VD)]]
def aggregateMessages[Msg: ClassTag](
    sendMsg: EdgeContext[VD, ED, Msg] => Unit,
    mergeMsg: (Msg, Msg) => Msg,
    tripletFields: TripletFields = TripletFields.All)
  : VertexRDD[A]
```


Class Discussion

1. How many airports are there?
   - In our graph, what represents airports?
   - Which operator could you use to find the number of airports?
2. How many routes are there?
   - In our graph, what represents routes?
   - Which operator could you use to find the number of routes?

How Many Airports are There?

How many airports are there?

- In our graph, what represents airports?

  Vertices

- Which operator could you use to find the number of airports?

  `graph.numVertices`

Pregel API

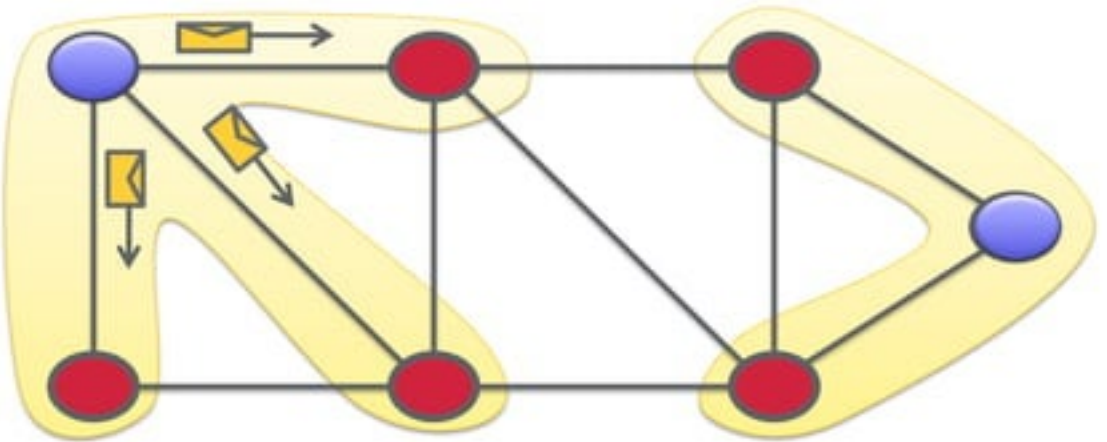
- GraphX exposes variant of Pregel API

- iterative graph processing
  - Iterations of message passing between vertices

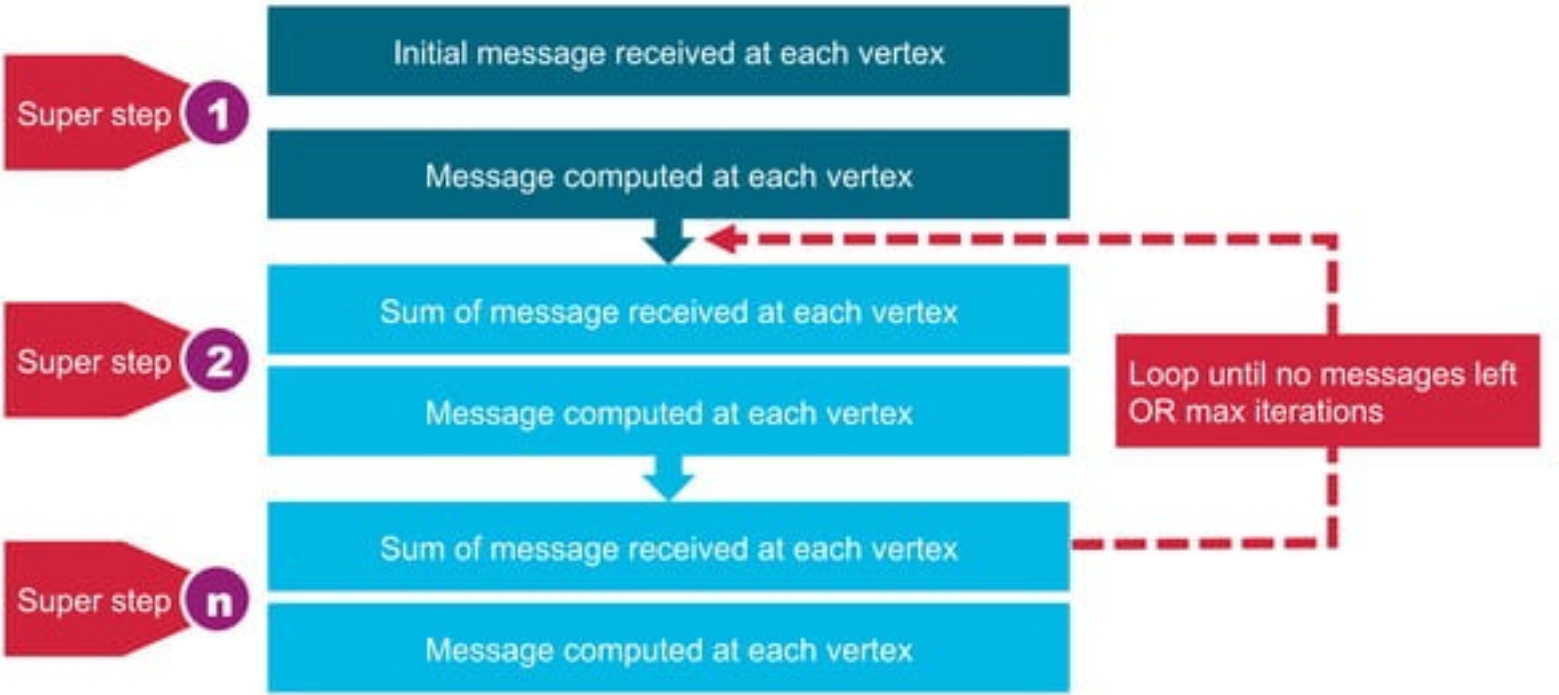
The Graph-Parallel Abstraction

A user-defined Vertex-Program runs on each Graph vertex

- Using messages (e.g. Pregel)
- Parallelism: run multiple vertex programs simultaneously



Pregel Operator



Pregel Operator: Example

Use Pregel to find the cheapest airfare:

```
// starting vertex
val sourceId: VertexId = 13024
// a graph with edges containing airfare cost calculation
val gg = graph.mapEdges(e => 50.toDouble + e.attr.toDouble/20 )
// initialize graph, all vertices except source have distance infinity
val initialGraph = gg.mapVertices((id, _) =>
   if (id == sourceId) 0.0 else Double.PositiveInfinity
```

Graph Class

Pregel

```
// Iterative graph-parallel computation
def pregel[A](initialMsg: A, maxIterations: Int, activeDirection: EdgeDirection)(
    vprog: (VertexID, VD, A) => VD,
    sendMsg: EdgeTriplet[VD, ED] => Iterator[(VertexID,A)],
    mergeMsg: (A, A) => A)
  : Graph[VD, ED]
```

Pregel Operator: Example

Use Pregel to find the cheapest airfare:

```
// call pregel on graph
val sssp = initialGraph.pregel(Double.PositiveInfinity)(
 // Vertex Program
 (id, distCost, newDistCost) => math.min(distCost, newDistCost),
 triplet => {
  // Send Message
  if (triplet.srcAttr + triplet.attr < triplet.dstAttr) {
   Iterator((triplet.dstId, triplet.srcAttr + triplet.attr))
  } else {
   Iterator.empty
  }
 },
 // Merge Message
 (a,b) => math.min(a,b)
)
```

Pregel Operator: Example

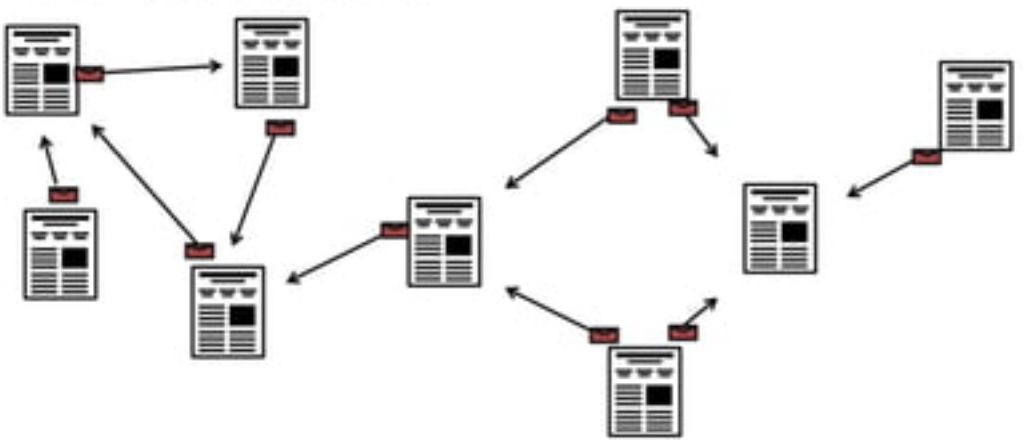
Use Pregel to find the cheapest airfare:

```
// routes , lowest flight cost
println(sssp.edges.take(4).mkString("\n"))
Edge(10135,10397,84.6)
Edge(10135,13930,82.7)
Edge(10140,10397,113.45)
Edge(10140,10821,133.5)
```

PageRank

```
graph.pageRank(tolerance).vertices
```

- Measures the importance of vertices in a graph
  - In links are votes
  - In links from important vertices are more important
- Returns a graph with vertex attributes



Page Rank: Example

Use Page Rank:

```
// use pageRank
val ranks = graph.pageRank(0.1).vertices
// join the ranks  with the map of airport id to name
val temp= ranks.join(airports)
temp.take(1)
// Array((15370,(0.5365013694244737,TUL)))

// sort by ranking
val temp2 = temp.sortBy(_._2._1, false)
temp2.take(2)
//Array((10397,(5.431032677813346,ATL)), (13930,(5.4148119418905765,ORD)))

// get just the airport names
val impAirports =temp2.map(_._2._2)
impAirports.take(4)
//res6: Array[String] = Array(ATL, ORD, DFW, DEN)
```

Use Case

Learn More

- https://www.mapr.com/blog/how-get-started-using-apache-spark-graphx-scala
- GraphX Programming Guide http://spark.apache.org/docs/latest/graphx-programming-guide.html
- MapR announces Free Complete Apache Spark Training and Developer Certification https://www.mapr.com/company/press-releases/mapr-unveils-free-complete-apache-spark-training-and-developer-certification
- Free Spark On Demand Training http://learn.mapr.com/?q=spark#-l
- Get Certified on Spark with MapR Spark Certification http://learn.mapr.com/?q=spark#certification-1,-l

MapR Converged Data Platform